DevOps
Assignment 4

CI/CD Pipelines with Jenkins and Groovy

Before Starting

- Work in the same group of 2 students from Assignment 3.
- Use the same private GitHub repository from Assignment 2 and 3. Create a new top-level folder **assignment-4/**.
- Reuse the AWS infrastructure provisioned by your Terraform code from Assignment 3. Do not create new infrastructure manually.
- Divide tasks equally and document each member's contribution in the README.
- Use proper Git workflow: one branch per task, merge to main via pull requests.
- Do not name folders after team members or roll numbers.
- Commit every Jenkinsfile, Groovy file, and Dockerfile to the repository. Do not paste pipeline code only into the Jenkins UI.

Objective

Build a working CI/CD pipeline using **Jenkins** and **Groovy** on top of the AWS infrastructure from Assignment 3. You will set up Jenkins, author a declarative pipeline that builds, tests, and deploys a sample application, write a small reusable Groovy shared library, integrate code quality with SonarQube, scan and push a Docker image to AWS ECR, drive Terraform from Jenkins, perform a Blue-Green deployment, and monitor the pipeline with Prometheus and Grafana.

Note:

This assignment must represent your own original work. **Copying Jenkinsfiles or Groovy libraries from online sources without understanding, sharing pipeline code between different groups, and use of AI tools to write Groovy or Jenkinsfile code without learning it yourself will all be treated as plagiarism.** During the viva, you will be asked to explain your Jenkinsfile stages, modify a stage live, and trace a failed build through the logs. If you cannot do this, you will receive zero marks regardless of the submitted report.

Task 1: Jenkins Installation and Basic Configuration

Requirements

- Provision an EC2 instance inside the VPC from Assignment 3 (via Terraform) that will host the **Jenkins controller**. Open port 8080 to your IP only and port 22 for SSH.
- Install Jenkins LTS using a **user_data** script. The script must also install Java 17, Git, Docker, the AWS CLI, and Terraform.
- Through the Jenkins UI, complete the initial setup wizard: create an **admin user** (do not keep the default initial password) and install the suggested plugins plus the following required plugins: *Pipeline*, *Git*, *GitHub Branch Source*, *Docker Pipeline*, *Credentials Binding*, *Pipeline Utility Steps*, *SonarQube Scanner*, *Blue Ocean*.
- Configure **one build agent** on a separate EC2 instance in a private subnet, connected to the controller via SSH. Label it `linux-agent`. Pipelines must run on the agent, not on the controller.
- Create the following **Jenkins Credentials** through the Jenkins UI, in the global domain: AWS access key + secret, GitHub personal access token, a SonarQube token (to be added in Task 4), Docker / ECR credentials, and a Slack webhook URL. Reference these credentials by ID from your pipelines. **No secret may appear in plaintext in any committed file.**
- Configure the **GitHub plugin** with your PAT so that Jenkins can clone your private repository and report status back to commits.
- In your repository, add a `jenkins/` folder containing: a **plugins.txt** listing all installed plugins, a short **setup.md** describing the controller and agent setup step by step, and your Terraform files for the controller and agent EC2 instances.

Deliverable

Screenshots of: the Jenkins dashboard after login, the *Manage Jenkins* > *Nodes* page showing your agent online, the *Manage Jenkins* > *Credentials* page listing all created credentials (IDs visible, values masked), and the *Manage Jenkins* > *Plugins* page showing the required plugins installed. A small sanity-check Pipeline job that runs `echo hello` on the agent and succeeds.

Task 2: Declarative Pipeline with Parallel Stages and Notifications

Requirements

- Build a small sample application of your choice (**Node.js + Express**, **Python Flask**, or **Spring Boot**) with: at least one REST endpoint (e.g., `/health`), at least **5 unit tests**, and at least **2 integration tests**. Put it under **app/** in your repository.
- Write a **Jenkinsfile** (Declarative) at the repo root with the following stages in order: *Checkout*, *Build*, *Test*, *Package*, *Deploy*.

- The *Test* stage must use a **parallel** block to run **unit tests** and **integration tests** concurrently. Each parallel branch must publish its JUnit test report. If any branch fails, the whole pipeline must fail.
- Use an **agent** directive at the pipeline level to target the `linux-agent` label from Task 1. Use an **environment** block with `credentials()` to inject at least one secret (e.g., your Slack webhook) without exposing it.
- Add a **post** section at the pipeline level with at least three handlers: *always* (archive build artifacts), *success* (send a Slack or email notification with the build URL), and *failure* (send a Slack or email notification naming the failing stage).
- Configure the job as a **Multibranch Pipeline** so it automatically discovers branches and pull requests from your GitHub repo. Add a **GitHub webhook** so pushes trigger builds automatically.
- Demonstrate at least **one successful build** and **one intentionally failing build** (e.g., break a test on a feature branch) and screenshot both in your report.

Deliverable

Blue Ocean visualization of a successful pipeline run showing all stages. Screenshot of the parallel stage view inside the *Test* stage. JUnit test report page in Jenkins. Screenshot of the Slack (or email) message for both a success and a failure. Screenshot of the GitHub webhook and of a PR build with the Jenkins status check reported back on the PR page.

Task 3: Reusable Jenkins Shared Library in Groovy

Requirements

- Create a **separate GitHub repository** called **jenkins-shared-library** with the standard layout: **vars/** and **src/**.
- Register the library globally in Jenkins under *Manage Jenkins* > *System* > *Global Pipeline Libraries*. Set *Default version* to `main` and keep *Load implicitly* disabled, so pipelines must explicitly load it with `@Library('your-lib') _`.
- Implement at least **two Groovy classes** under `src/org/yourteam/`. Suggested: **NotificationService** (with `sendSlack(message)` and `sendEmail(to, subject, body)` methods) and **DockerHelper** (with `buildImage(name, tag)` and `pushImage(name, tag)` methods). Use proper Groovy OOP: constructor receives the Jenkins `script` object, plus fields and methods.
- Implement at least **three global variables** under `vars/` that pipelines can call as steps. Suggested: `notifySlack.groovy`, `buildAndPushImage.groovy`, and `runSonarScan.groovy`. Each must accept a **Map of parameters** and validate that required keys are present before using them.
- Refactor the Jenkinsfile from Task 2 so that the Slack notification in the **post** section is replaced by a call to your `notifySlack` step. Keep everything else the same so the comparison is easy.
- Commit a **README.md** to the shared library repo explaining what each var and class does, with a minimal usage example.

Deliverable

Link to the shared library repository. Screenshot of the library registered in Jenkins under *Manage Jenkins* > *System*. A build log excerpt showing the line *Loading library your-lib@main*. Before/after diff of the Jenkinsfile showing the refactor to use `notifySlack`.

Task 4: Code Quality with SonarQube Integration

Requirements

- Deploy a **SonarQube server** on a small EC2 instance (t3.small or larger; SonarQube will not start on t3.micro) in your VPC. Use a **docker run** command or a simple **docker-compose.yml** inside the user_data script. The embedded H2 database is acceptable for this assignment.
- Open port 9000 on the SonarQube Security Group to **your IP only** and to the Jenkins agent Security Group. Do not expose it to the internet.
- Log into SonarQube, generate a **project token**, and add it to Jenkins as a *Secret text* credential. Register the SonarQube server under *Manage Jenkins* > *System* > *SonarQube servers*.
- Add a **Static Analysis** stage to your Jenkinsfile (before *Test*) that runs the SonarQube Scanner against your application and publishes the results. Use the `withSonarQubeEnv` step.
- After the scanner completes, use the `waitForQualityGate` step to block the pipeline until SonarQube returns the Quality Gate result. If the gate fails, the pipeline must fail. Configure the SonarQube webhook pointing back to Jenkins so this does not time out.
- Use the default **Sonar way** Quality Gate, but tighten at least **one condition**: require *Coverage on New Code ≥ 70%*. Take a screenshot of the Quality Gate configuration.
- Measure code coverage with a language-appropriate tool (nyc/Istanbul for Node, coverage.py for Python, JaCoCo for Java) and pass the coverage report to SonarQube so it appears on the project dashboard.

Deliverable

SonarQube project dashboard screenshot showing at least the Reliability, Security, Maintainability, and Coverage widgets. Screenshot of the Quality Gate configuration with your edited coverage condition. Jenkins console log excerpt showing the `waitForQualityGate` step returning *OK*. A second screenshot where the gate fails (intentionally lower coverage or add a bug) and blocks the pipeline.

Task 5: Docker Build, Vulnerability Scanning, and ECR Push

Requirements

- Write a **multi-stage Dockerfile** for your application with a distinct *builder* and a minimal *runtime* stage. The final image must run as a **non-root user** and must not include build tools like compilers or package caches.
- Add a **Container Build** stage in your Jenkinsfile that builds the image and tags it with **two tags**: the short Git commit SHA (e.g., `myapp:a1b2c3d`) and the branch name (e.g., `myapp:main`).
- Provision an **AWS ECR** private repository using Terraform. Add a simple lifecycle policy: keep the 10 most recent images and expire untagged images after 7 days.
- Authenticate to ECR from the Jenkins agent using an **IAM role** attached to the agent EC2 instance (not long-lived access keys). Document the IAM policy attached to the role.
- Add a **Security Scan** stage that runs **Trivy** against the built image. Configure Trivy to fail the pipeline on any **HIGH** or **CRITICAL** CVE that has an available fix. Maintain a `.trivyignore` file committed to the repo with justification comments for any CVEs you choose to ignore.

- If the scan passes, push both tags to ECR in a **Push** stage. Archive the Trivy report as a Jenkins artifact regardless of scan result.
- Demonstrate at least **one scan failure** (e.g., pin an older base image or an outdated dependency version) and then a passing scan after the fix. Both must appear in the report.

Deliverable

Dockerfile with highlighted multi-stage structure. ECR console screenshot showing both tags and the lifecycle policy. IAM role screenshot showing the policy attached to the agent. Jenkins console logs of a failing Trivy scan and a successful one. Trivy HTML or table report archived from a passing build.

Task 6: Terraform CI/CD Pipeline

Requirements

- Create a **separate parameterized Jenkins pipeline job** named `infra-pipeline` that manages your Terraform code from Assignment 3. Parameters: `ACTION` (choice: plan/apply/destroy) and `AUTO_APPROVE` (boolean, default false).
- Pipeline stages in order: *Checkout* → *Fmt & Validate* → *Security Scan (tfsec)* → *Plan* → *Manual Approval* → *Apply/Destroy*.
- The *Fmt & Validate* stage must run `terraform fmt -check` and `terraform validate` and fail the build if formatting is wrong or validation fails.
- The *Security Scan* stage must run **tfsec** against your Terraform code. Fail the build on any HIGH finding. Archive the tfsec report as an artifact.
- The *Plan* output must be **saved as a binary plan file** (e.g., `tfplan`), archived as a Jenkins artifact, and fed exactly into the *Apply* stage (`terraform apply tfplan`). This guarantees the apply matches the plan the user approved.
- The *Manual Approval* stage uses the **input** step with a 30-minute timeout. It must only appear when `ACTION=apply` or `ACTION=destroy` and `AUTO_APPROVE=false`. Show the approval prompt in a screenshot.
- Store the Terraform state in the **S3 backend** you configured in Assignment 3 (Task 3), with the **DynamoDB table** for state locking. Demonstrate locking by showing two concurrent runs where the second one waits or fails.

Deliverable

Jenkins job configuration screenshot showing the parameter form. Console output of every stage (fmt, validate, tfsec, plan, apply). Screenshot of the *Manual Approval* prompt. tfsec report archived as an artifact. Screenshot of the DynamoDB lock entry appearing during a running apply, and proof that a second concurrent run was blocked.

Task 7: Blue-Green Deployment to AWS

Requirements

- Extend the AWS infrastructure from Assignment 3 (via Terraform) to support a **Blue-Green** deployment: **two Auto Scaling Groups** (`asg-blue`, `asg-green`) behind the same Application Load Balancer, each attached to its own target group (`tg-blue`, `tg-green`).
- The ALB listener must have a **single forward rule** pointing to whichever target group is currently *live*. At any moment, exactly one of the two colors receives 100% of the traffic; the other is idle and available for the next deployment.

- Build a **Deploy-Production** stage in your Jenkinsfile that, when running on the `main` branch: (i) queries the ALB listener with the AWS CLI to determine which color is currently live, (ii) updates the *idle* ASG to use the new application image from ECR (e.g., update the launch template version and trigger an instance refresh), (iii) **waits for all targets in the idle target group to become healthy**, (iv) runs a smoke test by curling the idle target group through a second listener rule on a different port (used only for testing).
- If the smoke test passes, **switch the ALB listener** to forward to the previously idle target group. The previously live color becomes the new idle / rollback color.
- If the smoke test fails, the pipeline must **not switch the listener** and must fail the build with a clear message.
- Implement a **manual rollback job**: a separate Jenkins pipeline that accepts no parameters and simply flips the ALB listener back to the other target group. Useful if a problem is detected after traffic has been switched.
- Maintain a simple **deployment log file** in S3 (append JSON lines via `aws s3 cp`) recording each deployment with: timestamp, Git SHA, image tag, previous color, new color, and result (success / failed / rolled-back).

Deliverable

Architecture diagram of your Blue-Green setup on top of the Assignment 3 VPC. Screenshots of both target groups with healthy targets. Screenshot of the ALB listener rule before and after a switch. Jenkins console output of a successful deployment and of a failed smoke test that did NOT switch. Screenshot of the manual rollback job flipping traffic back. Contents of the S3 deployment log showing at least three entries.

Submission Guidelines

Report Format

- The final report must be submitted in **PDF format only**. Word documents will not be accepted.
- The report must be neatly formatted with clear headings, subheadings, and figure labels.
- Every screenshot must be followed by a written explanation (do not submit raw output without interpretation).
- Use proper figure captions, e.g., *Figure 5: Trivy scan failing on a HIGH CVE in the Jenkins console.*
- Include an **Architecture Diagram** at the beginning of the report showing the Jenkins controller, agent, SonarQube, Prometheus/Grafana, ECR, and the AWS infrastructure from Assignment 3, with arrows showing how a commit flows through to deployment.

Code Submission

- Submit all Jenkinsfiles, Groovy files, Dockerfiles, Terraform files, and Grafana dashboard JSON in a ZIP archive.
- Suggested folder structure: `app/`, `jenkins/` (plugins.txt, setup.md, controller + agent Terraform), `pipelines/` (Jenkinsfiles), `terraform/` (infrastructure from Assignment 3 plus Blue-Green additions), `observability/` (docker-compose, dashboard JSON, runbook.md). The shared library lives in its own separate repository; include a link in the README.
- Do **NOT** include `.terraform/` directories, `*.tfstate`, Jenkins secrets, AWS credentials, `.env` files, or any private keys. **Any credential committed to the repository will result in zero on Task 1 at minimum.**
- Include a **README.md** at the repository root explaining: prerequisites, how to bring up Jenkins and the supporting services, how to run each pipeline, how to tear everything down,

and a per-member contribution table.

- Tag the final submission commit as `assignment-4-final` on the `main` branch

Marks Distribution

Task	Title	Marks
1	Jenkins Installation and Basic Configuration	10
2	Declarative Pipeline with Parallel Stages and Notifications	15
3	Reusable Jenkins Shared Library in Groovy	15
4	Code Quality with SonarQube Integration	15
5	Docker Build, Vulnerability Scanning, and ECR Push	15
6	Terraform CI/CD Pipeline	15
7	Blue-Green Deployment to AWS	15
	Total	**100**